DAY 6 · STUDY GUIDE

Phase 6
n8n Automation

Webhook trigger · HTTP Request node · host.docker.internal · Binary field mapping · Respond to Webhook · Docker networking

> **Today's win in one line:** n8n now drives your backend as an automation. A three-node workflow — **Webhook → HTTP Request → Respond to Webhook** — exposes a URL that accepts a PDF, forwards it to your FastAPI `/ingest/pdf` endpoint across the Docker network boundary, and returns the ingest report to the caller. Proven end-to-end: a `curl` to the webhook came back with `pages_processed: 10, chunks_created: 10` — the report generated by FastAPI, relayed through n8n. This satisfies the roadmap's "implement something with n8n" requirement with a real trigger-to-action pipeline.

What you built / verified	How
Docker network path verified	`docker exec n8n wget -qO- http://host.docker.internal:8000/health` returned `{"status":"ok"}` — container can reach host FastAPI.
Webhook trigger node	POST method, path `ingest-pdf`, set to respond via the "Respond to Webhook" node (not immediately).
HTTP Request node	POST to `http://host.docker.internal:8000/ingest/pdf`, Form-Data body, n8n Binary File type.
Binary field mapping fixed	Input Data Field Name set to `file` (what curl labeled it), Name `file` (what FastAPI expects). Fixed a "no binary field 'data'" error.
Respond to Webhook node	Returns the first incoming item — the FastAPI report — back to the original caller.
End-to-end run	curl to `webhook-test/ingest-pdf` with the Day 1 PDF returned the full ingest report; all three nodes green.
Evidence saved + repo cleaned	Two screenshots committed; stopped tracking `n8n_data/` runtime files that were cluttering git status.

1 What n8n is and why it fits here

1.1 n8n in plain words

n8n is a workflow automation tool — think "visual glue between services." You build workflows by connecting nodes on a canvas: a trigger node starts things, action nodes do work, and data flows left to right between them. It's the open-source, self-hostable cousin of Zapier or Make.

DEFINITION	A node-based automation platform where each node is a step (trigger, HTTP call, transform, respond) and connections define data flow.
WHERE USED	Connecting apps without writing glue code: "when a form is submitted, save to a sheet and send a Slack message." Here: "when a PDF hits this URL, ingest it into my RAG engine."
EXAMPLE	Your workflow: Webhook (trigger) → HTTP Request (action) → Respond to Webhook (reply).
KEY TERMS	Node, trigger, workflow, canvas, execution, webhook, binary property.
WHY FOR INTERNSHIP	The roadmap explicitly names n8n. And in the ingestion-engine plan, n8n is the intended automation layer that triggers ingestion — so this isn't a throwaway demo, it's part of the real architecture.

1.2 Why a webhook trigger specifically

A webhook is just a URL that waits for an incoming HTTP request. It's the simplest way to let *anything* — a form, another service, a curl command, a different app — kick off your workflow. That makes your ingestion pipeline callable by the outside world without exposing FastAPI directly.

> **THE MENTAL MODEL**
>
> The webhook is n8n's "front door." Your FastAPI stays private on the host; n8n sits in front and decides what's allowed to trigger an ingest. In a real deployment you'd add auth, rate limits, or notifications around it — all as more nodes.

2 The Docker networking boundary

2.1 Why localhost doesn't work from a container

This is the single most important concept from today. Your n8n runs *inside a Docker container*. Your FastAPI runs *on the Windows host*. Inside a container, `localhost` refers to the container itself — not your machine. So an HTTP Request to `http://localhost:8000` would look inside the container, find no FastAPI, and fail with connection refused.

DEFINITION Each container has its own network namespace. Its `localhost` is isolated from the host's `localhost`.

THE FIX `host.docker.internal` — a special hostname Docker Desktop provides that resolves to the host machine from inside a container.

EXAMPLE From n8n, your FastAPI is `http://host.docker.internal:8000`. You'd seen this before — it's the same pattern as `OLLAMA_BASE_URL` in your `.env`.

KEY TERMS Container, network namespace, host, `host.docker.internal`, port mapping.

WHY FOR INTERNSHIP This trips up almost everyone who runs services partly in Docker and partly on the host. Explaining it cleanly signals real hands-on Docker experience.

2.2 How you verified it before building

Rather than build the whole workflow and hope, you tested the network path in isolation first:

```
docker exec n8n wget -qO- http://host.docker.internal:8000/health
```

`docker exec n8n` runs a command *inside* the n8n container. `wget` fetches the URL. So this is literally the container reaching out to the host FastAPI — the exact path the workflow uses. It returned `{"status":"ok"}`.

> **YOU PROVED THIS TODAY**
>
> Testing the risky dependency (cross-boundary networking) in one command before touching the canvas meant that when the workflow failed later, you already knew the network wasn't the cause. That's how you avoid debugging two unknowns at once.

3 The three nodes

3.1 What each node does

Node	Role	Key settings
Webhook	Trigger — exposes a URL, waits for a POST.	POST, path `ingest-pdf`, respond via Respond node.
HTTP Request	Action — forwards the file to FastAPI.	POST, URL `host.docker.internal:8000/ingest/pdf`, Form-Data, n8n Binary File.
Respond to Webhook	Reply — sends FastAPI's report back to the caller.	Respond with first incoming item.

3.2 Why "Respond to Webhook" is a separate node

By default a webhook replies immediately with a generic acknowledgment. But you want the caller to get *FastAPI's actual report* — which doesn't exist until the HTTP Request node finishes. So you set the webhook to "respond using the Respond to Webhook node," which holds the reply open until your last node produces the real answer, then sends it.

> **THE PATTERN**
>
> Trigger → do work → respond-with-result is the standard shape for a synchronous webhook that returns something meaningful. Without the dedicated respond node, the caller would get an empty "received" and never see the ingest report.

4 The binary field bug

4.1 The error

First run failed: **"The item has no binary field 'data' [item 0]."** The HTTP Request node was looking for a binary property called `data`, but the incoming file was stored under a property called `file`.

THE CAUSE curl sent the file with `-F "file=@..."`, so n8n's webhook stored the binary under the name `file`. But the HTTP Request node's "Input Data Field Name" was set to `data` (n8n's usual default).

THE CLUE n8n's INPUT panel showed the binary labeled `file`, not `data` — the mismatch was visible right there.

THE FIX Set "Input Data Field Name" to `file` so the node grabs the binary that actually arrived.

4.2 The two "file" fields — which is which

After the fix, two settings both said `file`, which is confusing. They mean different things:

- **Input Data Field Name = `file`** — "which binary coming *in* from the webhook do I pick up?" (curl named it `file`)
- **Name = `file`** — "what do I call this field when I send it *out* to FastAPI?" (FastAPI's parameter is `file: UploadFile`)

> **TRAP**
>
> These matching by coincidence hides that they're two independent settings. If curl had sent `-F "document=@..."`, the Input field would be `document` but Name would still be `file`. Knowing the difference is knowing how data flows through the node.

5 Repo hygiene: untracking runtime data

5.1 The n8n_data noise

After committing, `git status` kept showing `n8n_data/database.sqlite-wal` and `n8nEventLog.log` as modified. Those are n8n's internal database and logs — they change every time n8n runs, they're machine-specific, and they don't belong in the repo. They were committed before `.gitignore` covered them, so git kept tracking them.

THE FIX — `git rm -r --cached n8n_data` — stops git tracking the folder but keeps the files on disk, so n8n keeps working.

WHY --CACHED — Plain `git rm` deletes the files. `--cached` removes them from git's index only. Since `n8n_data/` is already in `.gitignore`, once untracked it stays ignored.

> **THE LESSON**
>
> .gitignore only prevents *new* files from being tracked. It does nothing for files already committed. To stop tracking something that slipped in early, you need `git rm --cached` plus the ignore rule.

6 How it all fits the ingestion engine

6.1 n8n's place in the architecture

The build plan always intended n8n as the automation front door: an external trigger (a webhook, or later a scheduled job or a file-drop watcher) that calls your FastAPI ingestion endpoints. Today you built the simplest version of that — a manual webhook. The same pattern extends to "watch a folder, ingest new PDFs automatically" or "on form submit, ingest and notify Slack."

> **YOU PROVED THIS TODAY**
>
> Your backend is now callable by automation, not just by a human typing curl. That's the difference between a script and a system: a system can be triggered by other systems.

Q&A Likely interview questions on today's work

Answers in your voice — the way you'd actually explain it, not a textbook.

Question	Your answer
What is n8n and what did you use it for?	It's a workflow automation tool — you build pipelines by connecting nodes on a canvas instead of writing glue code. I used it to trigger my ingestion backend: a webhook receives a PDF, forwards it to my FastAPI ingest endpoint, and returns the result. So my engine can be driven by automation, not just by me typing curl.
Walk me through your workflow's three nodes.	Webhook, HTTP Request, Respond to Webhook. The webhook exposes a URL and waits for a POST with a file. The HTTP Request node forwards that file to my FastAPI at host.docker.internal:8000/ingest/pdf. FastAPI ingests it and returns a report. The Respond to Webhook node sends that report back to whoever called the webhook.
Why host.docker.internal instead of localhost?	Because n8n runs inside a Docker container and FastAPI runs on my host machine. Inside a container, localhost means the container itself, not the host — so localhost:8000 would fail. host.docker.internal is a special hostname Docker Desktop gives containers to reach the host. Same reason my Ollama URL uses it.
How did you know the networking would work before building?	I tested it in isolation first. I ran docker exec n8n wget to the FastAPI health endpoint through host.docker.internal — that runs the fetch from inside the n8n container, exactly the path the workflow uses. It returned status ok, so I knew the network was fine before I touched the canvas. That way when something failed later, I'd already ruled the network out.
Why is "Respond to Webhook" a separate node?	By default a webhook replies immediately with a generic acknowledgment. But I want the caller to get FastAPI's actual ingest report, which doesn't exist until the HTTP Request node finishes. Setting the webhook to respond via the dedicated node holds the reply open until my last node produces the real result, then sends it.
You hit a binary field error — what was it?	First run failed with "no binary field 'data'." curl sent the file as -F "file=@...", so n8n stored the binary under the name "file," but my HTTP Request node was looking for one called "data," which is n8n's default. I changed the Input Data Field Name to "file" so it grabbed the binary that actually arrived.
You had two fields both set to "file" — what's the difference?	One is Input Data Field Name — which incoming binary the node picks up, named "file" because that's what curl called it. The other is Name — what the field is called when sent out to FastAPI, which must be "file" to match my file: UploadFile parameter. They matched by coincidence; if curl had named it "document," the input field would be "document" but the output name would still be "file."

Question	Your answer
How is this different from just calling FastAPI directly?	Functionally the ingest is the same, but n8n adds an automation layer in front. My FastAPI stays private on the host; n8n is the front door that external things can trigger. And it's extensible — I can add nodes for auth, notifications, or a folder-watch trigger without changing my backend. It turns a script into something other systems can drive.
Why did git keep showing n8n_data as modified?	n8n_data is n8n's internal database and logs — they change every run and are machine-specific. They'd been committed before my gitignore covered them, so git kept tracking them despite the ignore rule. I ran git rm --cached to stop tracking them while keeping the files on disk, and since they're in gitignore they stay ignored now.
What does gitignore actually do, then?	It only stops new, untracked files from being added. It does nothing for files already committed. So if something slips into git before you ignore it, you have to untrack it explicitly with git rm --cached, then the ignore rule takes over going forward.
Why self-host n8n in Docker instead of using n8n cloud?	Control. Self-hosting lets me set environment variables, control webhook payload limits, keep data local, and run it right next to my other Docker services like Qdrant. For an ingestion pipeline that handles files, keeping everything on my own infrastructure matters — nothing leaves the machine.
How would you extend this workflow?	A few ways. Swap the manual webhook for a folder-watch or schedule trigger so ingestion happens automatically. Add a notification node so I get a Slack or email when a file is ingested. Add error handling so a failed ingest alerts me instead of failing silently. Add auth on the webhook so not just anyone can trigger it.
What's the security concern with an open webhook?	Anyone who knows the URL can trigger an ingest and push data into my vector store. In production I'd add authentication — a header token or n8n's built-in webhook auth — plus rate limiting and a payload size cap. Right now it's an internal demo, so it's open, but I'd lock it down before exposing it.
Walk me through the full path of one request.	curl POSTs a PDF to the n8n webhook URL. The Webhook node receives it and stores the file as a binary named "file." The HTTP Request node picks up that binary and POSTs it as multipart form-data to host.docker.internal:8000/ingest/pdf. FastAPI validates, extracts text, chunks, embeds, and upserts to Qdrant, then returns a JSON report. The Respond to Webhook node takes that report and sends it back as the webhook's response, so curl prints the ingest result.

Day 6 · Phase 6 complete. n8n webhook workflow driving FastAPI ingestion end-to-end across the Docker network boundary. Remaining engine work: Phase 7 (polish — logging, content-hash dedup, README, demo). Roadmap items still open and highest-priority: OpenCV/DeepFace face app, ReAct agent from the 1-hour tutorial.